

```

/**
 * components/ui/stat-card.tsx
 *
 * Unified stat display card. Replaces the repeated pattern of
 * [icon] [label] [large number] [sub-label] [badge] across every
 * analytics mode and the AI health dashboard.
 *
 * Usage:
 *   <StatCard
 *     label="Critical Alerts"
 *     value={criticalCount}
 *     icon={AlertTriangle}
 *     variant="critical"
 *     subLabel="Require immediate attention"
 *   />
 */

import { type LucideIcon, TrendingUp, TrendingDown, Minus } from 'lucide-react';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Badge } from '@components/ui/badge';
import { Skeleton } from '@components/ui/skeleton';
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from '@components/ui/tooltip';
import { cn } from '@lib/utils';

// — Types —————

export type StatVariant = 'neutral' | 'positive' | 'warning' | 'critical';

export interface StatCardProps {
  label: string;
  value: string | number | null;
  subLabel?: string;
  icon?: LucideIcon;
  variant?: StatVariant;
  /** Show a percentage trend vs previous period */
  trend?: {
    value: number;
    /** e.g. "vs last month" */
    label?: string;
  };
}

```

```

    /** For costs: positive trend is bad. Invert to flip coloring. */
    invertColor?: boolean;
  };
  /** Small badge text next to the value */
  badge?: string;
  /** Tooltip explaining what this metric means */
  helpText?: string;
  isLoading?: boolean;
  className?: string;
  'data-testid'?: string;
}

// — Variant config —————

const variantConfig: Record<
  StatVariant,
  { value: string; iconBg: string; iconColor: string }
> = {
  neutral: {
    value:      'text-foreground',
    iconBg:     'bg-muted',
    iconColor:  'text-muted-foreground',
  },
  positive: {
    value:      'text-emerald-600 dark:text-emerald-400',
    iconBg:     'bg-emerald-50 dark:bg-emerald-950',
    iconColor:  'text-emerald-600 dark:text-emerald-400',
  },
  warning: {
    value:      'text-amber-600 dark:text-amber-400',
    iconBg:     'bg-amber-50 dark:bg-amber-950',
    iconColor:  'text-amber-600 dark:text-amber-400',
  },
  critical: {
    value:      'text-red-600 dark:text-red-400',
    iconBg:     'bg-red-50 dark:bg-red-950',
    iconColor:  'text-red-600 dark:text-red-400',
  },
};

// — Component —————

export function StatCard({
  label,
  value,
  subLabel,
  icon: Icon,

```

```

variant = 'neutral',
trend,
badge,
helpText,
isLoading = false,
className,
'data-testid': testId,
}: StatCardProps) {
  const cfg = variantConfig[variant];

  // Determine trend direction + color
  let TrendIcon: LucideIcon | null = null;
  let trendColor = '';
  if (trend) {
    const isPositiveChange = trend.value > 0;
    const isGoodChange = trend.invertColor ? !isPositiveChange : isPositiveChange
    TrendIcon = trend.value > 0 ? TrendingUp : trend.value < 0 ? TrendingDown : M
    trendColor = isGoodChange
      ? 'text-emerald-600 dark:text-emerald-400'
      : 'text-red-600 dark:text-red-400';
  }

  const card = (
    <Card className={cn('', className)} data-testid={testId}>
      <CardHeader className="flex flex-row items-center justify-between space-y-0">
        <CardTitle className="text-sm font-medium text-muted-foreground">
          {label}
        </CardTitle>
        {Icon && (
          <div className={cn('p-1.5 rounded-md', cfg.iconBg)}>
            <Icon className={cn('h-4 w-4', cfg.iconColor)} />
          </div>
        )}
      </CardHeader>

      <CardContent>
        {isLoading ? (
          <div className="space-y-2">
            <Skeleton className="h-8 w-24" />
            <Skeleton className="h-3 w-16" />
          </div>
        ) : (
          <>
            <div className="flex items-baseline gap-2 flex-wrap">
              <span className={cn('text-3xl font-bold', cfg.value)}>
                {value ?? '-'}
              </span>
            </div>
          </>
        )}
      </CardContent>
    </Card>
  )
}

```

```

        {badge && (
            <Badge variant="secondary" className="text-xs h-5">
                {badge}
            </Badge>
        )}
    </div>

    {/* Trend + sub-label row */}
    {(trend || subLabel) && (
        <div className="flex items-center gap-2 mt-1.5 flex-wrap">
            {trend && TrendIcon && (
                <span
                    className={cn(
                        'flex items-center gap-0.5 text-xs font-medium',
                        trendColor,
                    )}
                >
                    <TrendIcon className="h-3 w-3" />
                    {Math.abs(trend.value).toFixed(1)}%
                    {trend.label && (
                        <span className="text-muted-foreground font-normal ml-1">
                            {trend.label}
                        </span>
                    )}
                </span>
            )}
            </span>
        )}
        {subLabel && (
            <p className="text-xs text-muted-foreground">{subLabel}</p>
        )}
    </div>
    )}
</>
)}
</CardContent>
</Card>
);

// Wrap in tooltip only when helpText is provided
if (!helpText) return card;

return (
    <TooltipProvider>
        <Tooltip>
            <TooltipTrigger asChild>{card}</TooltipTrigger>
            <TooltipContent className="max-w-xs">
                <p className="text-sm">{helpText}</p>
            </TooltipContent>
        </Tooltip>
    </TooltipProvider>
);

```

```
        </Tooltip>
    </TooltipProvider>
);
}
```